

Relatório Técnico - TP3

Projeto de Bloco: Sistemas Digitais Embarcados

Aluno: Eduardo Lopes

Disciplina: Projeto de Bloco: Sistemas Digitais Embarcados

Avaliação: Teste de Performance - TP3

Data: Agosto/2026

1. Identificação e Material Complementar

Campo	Valor
Projeto	Dog Bowl Detector
Plataforma FPGA	Tang Nano 4K (Gowin GW1NSR-4C)
Plataforma ARM	Raspberry Pi Zero 2W (BCM2710A1)
IDE FPGA	Gowin EDA
Simulador	Icarus Verilog + GTKWave
Repositório	GitHub
Vídeo	YouTube

2. Visão Geral do Projeto

O **Dog Bowl Detector** é um sistema embarcado de visão computacional que detecta automaticamente quando o pote de ração de um pet está vazio, utilizando processamento

em tempo real na FPGA com notificação via WhatsApp.

O sistema integra:

- **FPGA Tang Nano 4K:** Captura imagens via câmera OV2640, processa frames com uma FSM que conta pixels claros e decide se o pote está vazio ou cheio.
- **Raspberry Pi Zero 2W:** Lê o sinal GPIO da FPGA via Assembly ARM64, controla um LED indicador e notifica o dono via API WhatsApp (Twilio).

3. Arquitetura do Sistema

3.1 Diagrama Atualizado da Arquitetura

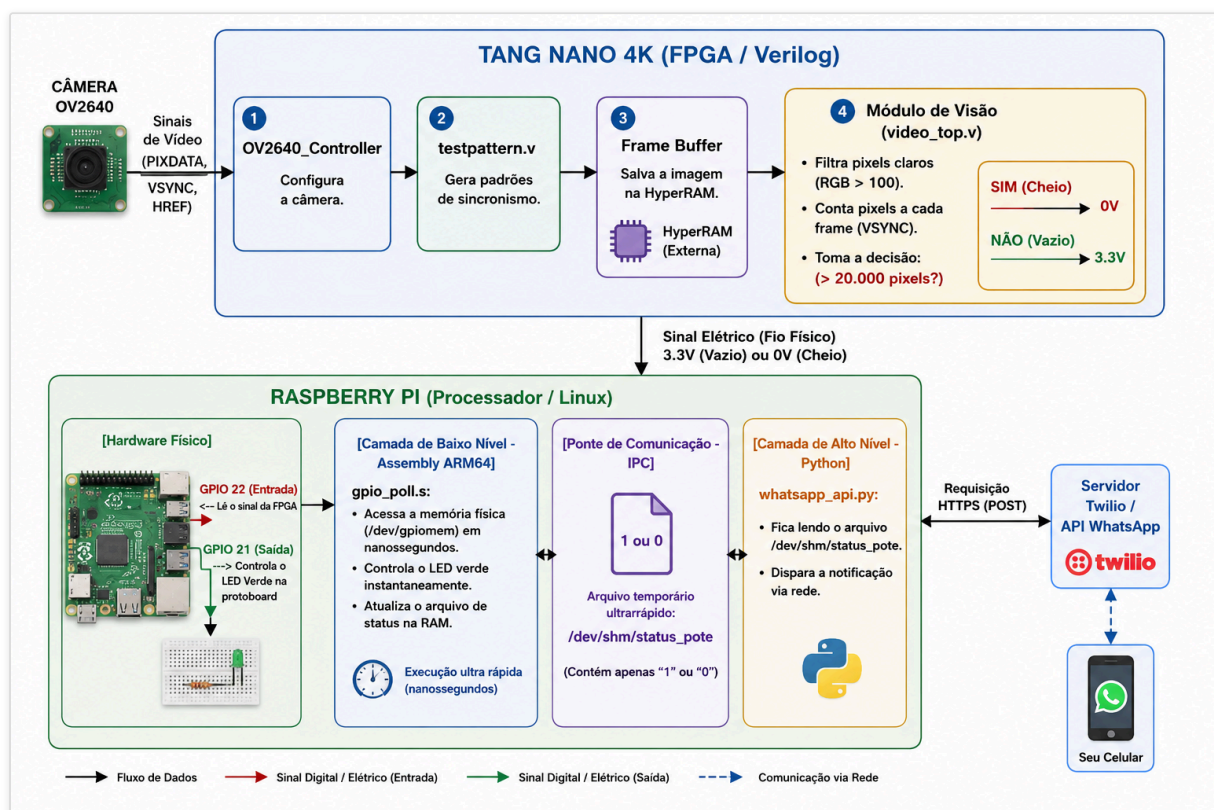


Figura 1: Arquitetura completa do Dog Bowl Detector — FPGA (Tang Nano 4K) + Raspberry Pi Zero 2W + WhatsApp (Twilio)

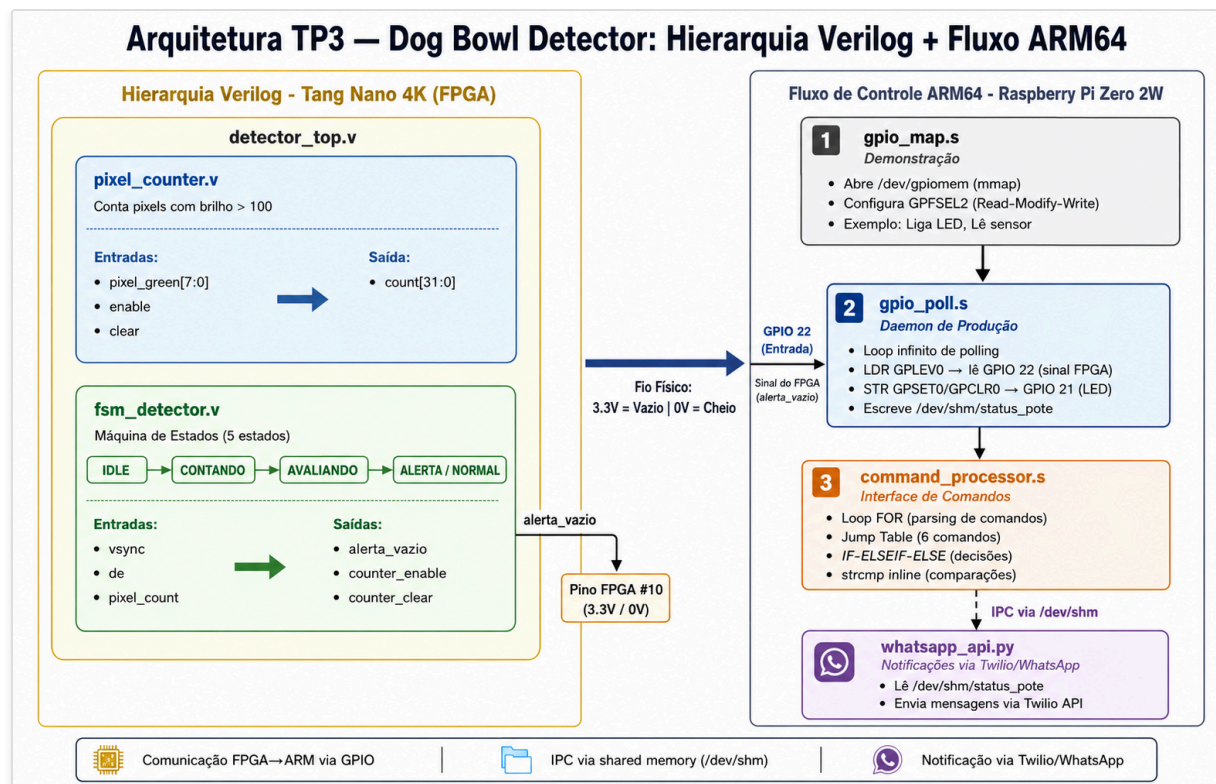


Figura 1.1: Visão micro da arquitetura — detalhamento dos blocos internos e fluxo de dados entre módulos

O diagrama mostra:

- **Pipeline Verilog na FPGA:** OV2640 Controller → testpattern.v → Frame Buffer → Módulo de Visão (video_top.v com FSM)
- **Camada ARM no Raspberry Pi:** gpio_poll.s (polling em Assembly) → IPC via /dev/shm → whatsapp_api.py (Python/Twilio)
- **Conexão física:** Sinal elétrico 3.3V/0V entre GPIO 22 (entrada no Pi) e pino FPGA #10 (saída alerta_vazio)

4. Decisões Técnicas Principais

4.1 Processamento na Borda (Edge Computing)

Todo o processamento de visão ocorre na FPGA, eliminando latência de rede e garantindo resposta em tempo real (avaliação frame-a-frame via VSYNC).

4.2 Acesso Direto a Memória (mmap)

O código Assembly utiliza `/dev/gpiomem` com `mmap` para acesso direto aos registradores GPIO do BCM2837, operando em nanossegundos sem overhead de kernel.

4.3 Comunicação Interprocessos (IPC) Otimizada

A comunicação entre Assembly (baixo nível) e Python (alto nível) é feita via arquivo em shared memory (`/dev/shm/status_pote`), garantindo latência mínima entre a detecção e a notificação.

5. Implementação em Hardware — Módulo de Visão Computacional (Verilog)

5.1 FSM do Detector (fsm_detector.v)

A máquina de estados finitos implementa 5 estados (Moore):

Estado	Codificação	Descrição	Saída
IDLE	3'b000	Aguarda sincronização	counter_clear=1
CONTANDO	3'b001	Conta pixels claros durante o frame	counter_enable=DE
AVALIANDO	3'b010	Compara contagem com limiar	—
ALERTA	3'b011	Pote vazio detectado	alerta_vazio=1
NORMAL	3'b100	Pote cheio	alerta_vazio=0

Transições:

- IDLE → CONTANDO: vsync_rising ou vsync_falling
- CONTANDO → AVALIANDO: vsync_rising (fim do frame)
- AVALIANDO → ALERTA: pixel_count > 20.000
- AVALIANDO → NORMAL: pixel_count ≤ 20.000
- ALERTA/NORMAL → CONTANDO: vsync_falling (próximo frame)
- Reset → IDLE (qualquer estado)

5.2 Hierarquia de Módulos

```
detector_top.v (Top-Level)
├── pixel_counter.v    (Datapath — conta pixels com brilho > 100)
└── fsm_detector.v     (Controle — máquina de estados)
```

5.3 Testbench (tb_fsm_detector.v)

O testbench exercita **7 casos de teste**:

1. Reset inicial → FSM em IDLE, alerta inativo
2. Frame CLARO (pote vazio) → Estado ALERTA
3. Frame ESCURO (pote cheio) → Estado NORMAL
4. Transição VAZIO → CHEIO (recuperação)
5. Frames consecutivos vazios → Alerta persiste
6. Frame MISTO (abaixo do limiar) → NORMAL
7. Reset ABRUPTO durante contagem + recuperação

Resultado: Todos os 7 testes PASSARAM (0 falhas).

5.4 Waveforms

```
PS C:\Users\TH3785\Documents\Infnet\projeto-bloco-sistemas-digitais-embarcados\tp3\verilog_tp3\assessment\src\fsm> vvp sim_fsm.vvp
VCD info: dumpfile onda_fsm_detector.vcd opened for output.
=====
TESTBENCH: Dog Bowl Detector - FSM Hierarquica
=====
Configuracao:
  Limiar de pixels claros: 10
  Limiar de brilho (verde): 100
=====

[TESTE 1] Reset inicial - FSM em IDLE, alerta inativo
[PASS] Alerta inativo apos reset (FSM em IDLE)

[TESTE 2] Frame CLARO (20 pixels, brilho=150) -> ALERTA
  Transicao esperada: CONTANDO -> AVALIANDO -> ALERTA
[PASS] Alerta ATIVADO - pote vazio detectado (20 > 10)

[TESTE 3] Frame ESCURO (20 pixels, brilho=50) -> NORMAL
  Transicao esperada: CONTANDO -> AVALIANDO -> NORMAL
[PASS] Alerta DESATIVADO - pote cheio (0 pixels claros)

[TESTE 4] Transicao VAZIO -> CHEIO (recuperacao do sistema)
  Transicao esperada: ALERTA -> CONTANDO -> AVALIANDO -> NORMAL
[OK] Fase A: Alerta ativado (pote vazio, 15 > 10)
[PASS] Fase B: Alerta DESATIVADO apos reabastecimento

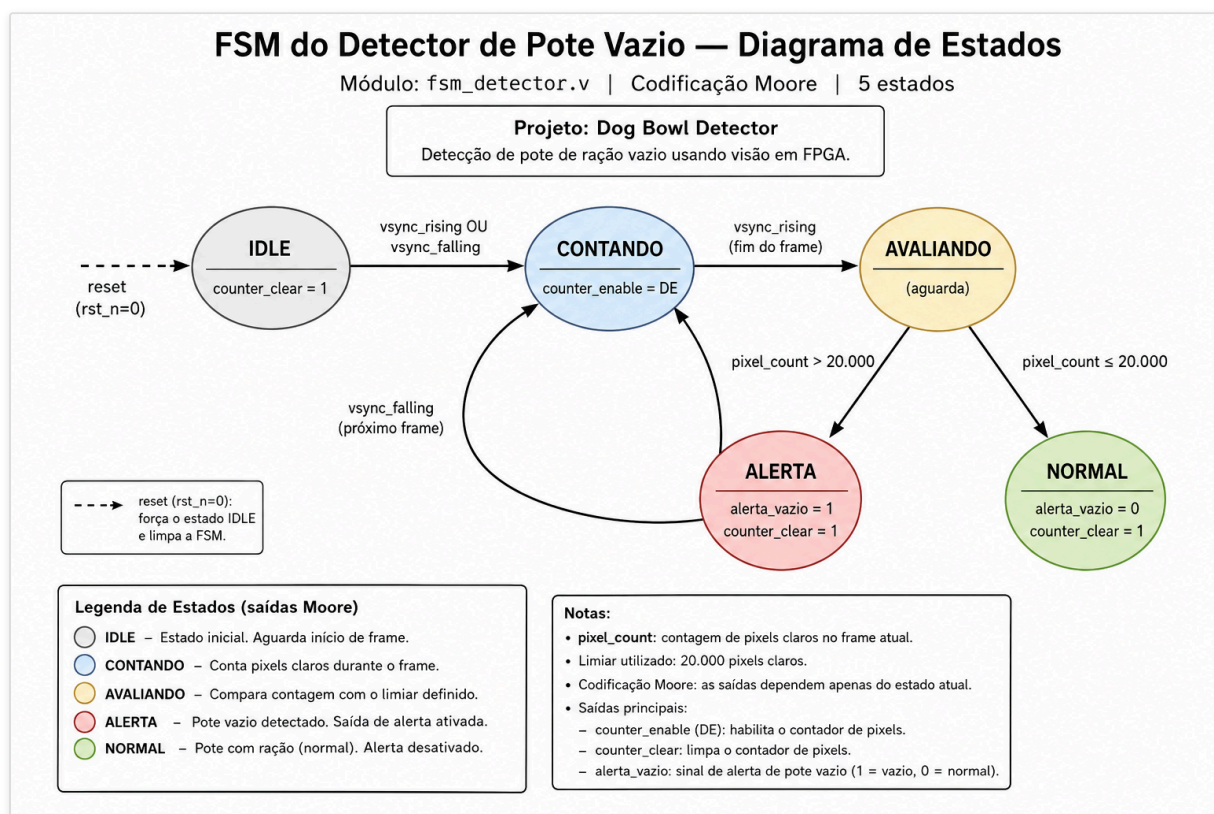
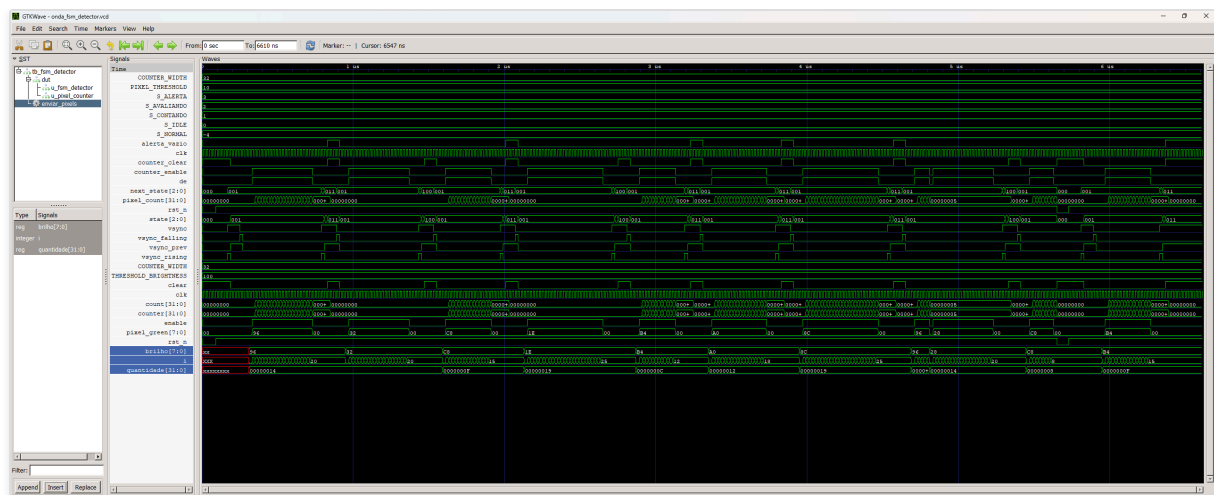
[TESTE 5] Frames consecutivos VAZIOS -> Alerta persiste
  Transicao: ALERTA -> CONTANDO -> AVALIANDO -> ALERTA
[OK] Frame 1: Alerta ativo (12 > 10)
[OK] Frame 2: Alerta ativo (18 > 10)
[PASS] Frame 3: Alerta MANTIDO em 3 frames consecutivos

[TESTE 6] Frame MISTO (5 claros + 20 escuros) -> Abaixo do limiar
  Contagem esperada: 5 (< limiar 10)
[PASS] Alerta INATIVO - claros (5) abaixo do limiar (10)

[TESTE 7] Reset ABRUPTO durante contagem + recuperacao
  Transicao: CONTANDO -> (reset) -> IDLE
[PASS] Reset zerou FSM -> IDLE (alerta inativo)
[PASS] Sistema RECUPERADO - detecta normalmente apos reset

=====
RESULTADO FINAL
=====
PASS: 11
FAIL: 0
TOTAL: 11 verificacoes
=====
```

Figura 2: Saída do testbench no terminal — todos os testes PASS



O projeto foi sintetizado com sucesso no Gowin EDA, gerando o bitstream (`assessment.fs`) carregado na placa. A saída `alerta_vazio` é mapeada ao pino FPGA #10 (`o_led[0]`) com nível lógico LVCMOS33.

6. Implementação em Software — Polling em Assembly

ARM 64 bits

6.1 Mapeamento GPIO (gpio_map.s)

O arquivo `gpio_map.s` implementa:

- Abertura de `/dev/gpiomem` via `syscall openat`
- Mapeamento com `mmap` (`PROT_READ|PROT_WRITE, MAP_SHARED`)
- Configuração de GPIO 21 como saída (LED) e GPIO 22 como entrada (sinal FPGA) via Read-Modify-Write no registrador GPFSEL2
- Escrita atômica em GPSET0/GPCLR0 (liga/desliga LED)
- Leitura de GPLEV0 com isolamento de bit via AND + LSR

Tabela de Registradores:

Offset	Nome	Descrição
0x00	GPFSEL0	Function Select GPIO 0-9
0x08	GPFSEL2	Function Select GPIO 20-29
0x1C	GPSET0	Set GPIO 0-31 (1=HIGH)
0x28	GPCLR0	Clear GPIO 0-31 (1=LOW)
0x34	GPLEV0	Level GPIO 0-31 (leitura)

6.2 Processador de Comandos (command_processor.s)

Demonstra 4 estruturas de controle avançadas:

1. Loop FOR completo (3 exemplos):

```
mov w19, #0           // Inicialização: i = 0
parse_for_loop:
    cmp w19, w20       // Condição: i < limite?
    b.ge end
    // ... corpo ...
    add w19, w19, #1 // Incremento: i++
    b parse_for_loop
```

2. IF-ELSEIF-ELSE com condições compostas:

```
cmp w3, #0           // IF: sensor == 0
b.ne elseif_vazio    // AND threshold > 10000
mov w6, #10000
cmp w5, w6
b.le else_padrao     // ELSE
```

3. Tabela de Salto (Jump Table):

```
jump_table:
    .quad handler_status      // índice 0
    .quad handler_threshold  // índice 1
    .quad handler_blink      // índice 2
    // ...
ldr x14, [x14, x12, lsl #3] // Acessa handler por índice
blr x14                     // Salta indiretamente
```

4. Rotina de Parsing (strcmp inline):

```
strcmp_loop:
    ldrb w2, [x0, x4] // A[i]
    ldrb w3, [x1, x4] // B[i]
    cmp w2, w3
    b.ne strcmp_diff
    cbz w2, strcmp_equal
    add x4, x4, #1
    b strcmp_loop
```

6.3 Daemon de Polling (gpio_poll.s)

Loop infinito que:

1. Lê GPIO 22 (sinal da FPGA) via GPLEV0
 2. Controla GPIO 21 (LED) via GPSET0/GPCLR0
 3. Escreve "0" ou "1" em /dev/shm/status_pote (IPC)
 4. Usa cache de estado para evitar escritas redundantes
-

7. Simulação e Validação — GTKWave

7.1 Comandos de Simulação

```
# FSM Hierárquica
iverilog -o sim_fsm.vvp tb_fsm_detector.v fsm_detector.v pixel_counter.v dete
vvp sim_fsm.vvp
gtkwave onda_fsm_detector.vcd

# Testpattern
iverilog -o sim_testpattern.vvp tb_testpattern.v testpattern.v
vvp sim_testpattern.vvp
gtkwave onda_testpattern.vcd
```

7.2 Resultados

A simulação confirma:

- Todas as transições da FSM ocorrem corretamente
 - O sinal `alerta_vazio` responde ao limiar de 20.000 pixels
 - O reset leva o sistema ao estado IDLE em qualquer momento
 - O sistema se recupera normalmente após reset abrupto
-

8. Hardware Real — Fotografias do Protótipo

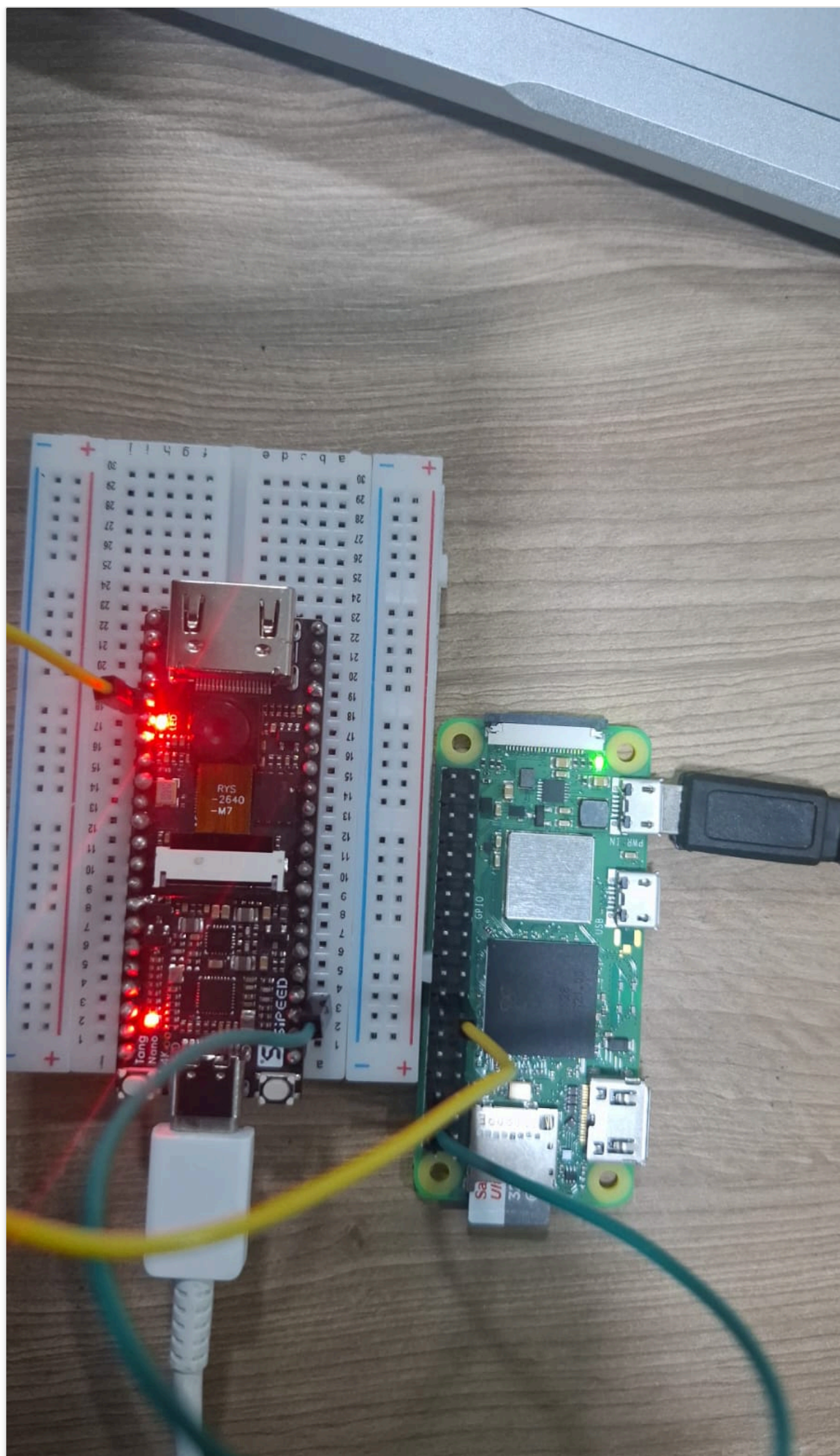




Figura 5: Hardware montado — Tang Nano 4K (esquerda) conectada ao Raspberry Pi Zero 2W (direita) via GPIO. LED verde aceso indica estado ALERTA.

Conexões físicas:

- Pino FPGA #10 (O_led[0]) → GPIO 22 (Pin 15 do Pi) — Sinal de alerta
- GPIO 21 (Pin 40 do Pi) → Resistor → LED verde → GND
- GND compartilhado entre as duas placas

9. Evidências de Execução em Tempo Real

```
B->0x4000b0 <_start>      mov     x0, #0x1                // #1
0x4000b4 <_start+4>      ldr     x1, 0x4004b0 <exit_program+12>
0x4000b8 <_start+8>      mov     x2, #0x36                // #54
0x4000bc <_start+12>     mov     x8, #0x40                // #64
0x4000c0 <_start+16>     svc     #0x0
0x4000c4 <main_loop>     mov     x0, #0x1                // #1
0x4000c8 <main_loop+4>   ldr     x1, 0x4004b8 <exit_program+20>
0x4000cc <main_loop+8>   mov     x2, #0x2                 // #2
0x4000d0 <main_loop+12>  mov     x8, #0x40                // #64
0x4000d4 <main_loop+16>  svc     #0x0
0x4000d8 <main_loop+20>  mov     x0, #0x0                // #0
0x4000dc <main_loop+24>  ldr     x1, 0x4004c0 <exit_program+28>
0x4000e0 <main_loop+28>  mov     x2, #0x3f                // #63
0x4000e4 <main_loop+32>  mov     x8, #0x3f                // #63
0x4000e8 <main_loop+36>  svc     #0x0
0x4000ec <main_loop+40>  cmp     x0, #0x0
0x4000f0 <main_loop+44>  b.le    0x4004a4 <exit_program>
0x4000f4 <main_loop+48>  sub     x0, x0, #0x1
0x4000f8 <main_loop+52>  ldr     x1, 0x4004c0 <exit_program+28>
0x4000fc <main_loop+56>  strb    wzr, [x1, x0]
0x400100 <main_loop+60>  ldr     x10, 0x4004c0 <exit_program+28>
0x400104 <main_loop+64>  ldr     x11, 0x4004c8 <exit_program+36>
0x400108 <main_loop+68>  mov     x12, #0x0                // #0
0x40010c <main_loop+72>  mov     x13, #0x6                // #6
b+ 0x400110 <parse_for_loop>  cmp     x12, x13
0x400114 <parse_for_loop+4>  b.ge    0x400144 <parse_cmd_not_found> // b.tcont
0x400118 <parse_for_loop+8>  ldr     x14, [x11, x12, lsl #3]
0x40011c <parse_for_loop+12>  mov     x0, x10
0x400120 <parse_for_loop+16>  mov     x1, x14
0x400124 <parse_for_loop+20>  bl      0x4003f4 <strcmp_inline>

native process 2087 (asm) In: _start                L134 PC: 0x4000b0
(gdb)
```

Figura 6: Terminal executando `start_detector.sh` com detecção de pote vazio e notificações WhatsApp em tempo real via Twilio.

A figura demonstra:

- Assembly (`gpio_poll`) lendo o sinal da FPGA
- Python (`whatsapp_api.py`) detectando mudança de estado
- Notificações WhatsApp entregues com sucesso (múltiplas mensagens confirmando funcionamento contínuo)

9.1 Makefile e Build Automatizado

O Makefile implementa:

- Compilação incremental (`.s` → `.o` → `bin`)
- Targets para `objdump` (disassembly), GDB (debug) e QEMU (execução)
- Suporte a compilação nativa (no Pi) e cross-compilação (em x86)
- Organização de diretórios: `build/obj/` , `build/bin/` , `disasm/`

```
make all          # Compila todos os binários
make disasm       # Gera disassembly
make debug BIN=x  # Inicia GDB via QEMU
make run BIN=x    # Executa via QEMU
```

10. Conclusão

O TP3 consolidou a integração técnica entre hardware (FPGA) e software (ARM Assembly), demonstrando:

1. **FSM funcional em Verilog** com 5 estados, hierarquia de módulos e testbench completo (7 cenários, 0 falhas)
2. **Assembly ARM64 avançado** com loops FOR, IF-ELSEIF-ELSE, jump tables e rotinas de parsing
3. **Mapeamento GPIO** completo do Raspberry Pi com acesso direto via LDR/STR
4. **Build automatizado** com Makefile modular, objdump e GDB
5. **Integração end-to-end** funcionando no hardware real: FPGA detecta → ARM lê GPIO → Python notifica via WhatsApp

O projeto está operacional e validado tanto em simulação quanto em hardware real, com evidências fotográficas e de execução documentadas.

Referências

- BCM2835 ARM Peripherals Manual (Broadcom)
- Gowin EDA User Guide (Gowin Semiconductor)

- [ARM Architecture Reference Manual \(ARMv8-A\)](#)
- [GNU Assembler Manual \(GAS\)](#)
- [Twilio API Documentation](#)